

Gestione Processi

Ogni processo ha un ID associato (**PID**): un intero positivo che identifica univocamente il processo nel sistema;

- la system call `getpid()` restituisce il PID del processo chiamante

```
#include <unistd.h>

pid_t getpid(void);
```

Ogni processo ha un genitore: il processo che lo ha creato.

- La system call `getppid()` permette ad un processo di conoscere il PID del proprio padre
- Le relazioni tra processi costituiscono una struttura ad albero.
 - Il genitore di ogni processo ha a sua volta un genitore...fino ad arrivare alla radice: il processo *init*

```
#include <unistd.h>

pid_t getppid(void);
```

Esempio: 01-test_pid.c

Grazie al comando `top` posso vedere l'elenco di TUTTI i processi creati.

Grazie al comando `ps` posso vedere l'elenco dei processi attualmente in esecuzione sulla macchina:

- Il comando `ps -aux` elenca tutti i processi in esecuzione,
 - con al comando `ps -aux | grep PID=<num>` posso selezionare un determinato processo dall'elenco.
 - Il comando `ps -axjf` elenca tutti i processi in esecuzione con una rappresentazione gerarchica (quale processo è figlio di chi).
-

Con il termine “Controllo dei Processi” si indica un insieme di operazioni che include la creazione di nuovi processi, l'esecuzione di processi, e la loro terminazione.

A queste operazioni corrispondono le system call: `fork()`, `exit()`, `wait()`, e `execve()`

In soldoni:

- Creazione dei Processi: `fork()`

La syscall `fork()` **permette a un processo, il padre, di creare un altro processo detto figlio.**

- Il figlio è (quasi!) una copia esatta del padre: ottiene copie di stack, data, heap, and text segments del padre.
- Il termine `fork` deriva dal fatto che ci si può raffigurare il processo padre come un processo che si suddivide in due.

- Terminazione dei Processi: `exit(status)`

La syscall `exit(status)` **termina un processo, rendendo le risorse utilizzate dal processo (memoria, descrittori dei file aperti, etc.) nuovamente disponibili per essere allocate dal kernel.**

- L'argomento `status` è un intero che descrive lo stato di terminazione del processo:
 - utilizzando la system call `wait()` il processo padre può risalire a tale `status`.

- Monitoraggio dei Processi: `wait(&status)`

La syscall `wait(&status)` ha due fini:

1. se un figlio non ha ancora concluso la propria esecuzione (chiamando una `exit()`), la `wait()` **sospende l'esecuzione del processo chiamante finché uno dei figli non ha terminato la propria esecuzione**; quindi, è uno strumento di sincronizzazione
2. dopo la terminazione del figlio, lo stato di terminazione del figlio è restituito nell'argomento `status` della `wait()`.

- Esecuzione dei Processi: `execve(pathname, argv, envp)`

La syscall `execve(pathname, argv, envp)` **carica un nuovo programma nella memoria del processo ...**

Vediamo ora più nel dettaglio queste operazioni:

Creazione dei Processi: fork()

La systemcall `fork()` permette a un processo, il padre, di crearne un altro detto figlio.

- Il figlio è (quasi!) una copia esatta del padre: ottiene copie di stack, data, heap, and text segments del padre.
- Il termine fork deriva dal fatto che ci si può raffigurare il processo padre come un processo che si suddivide in due.

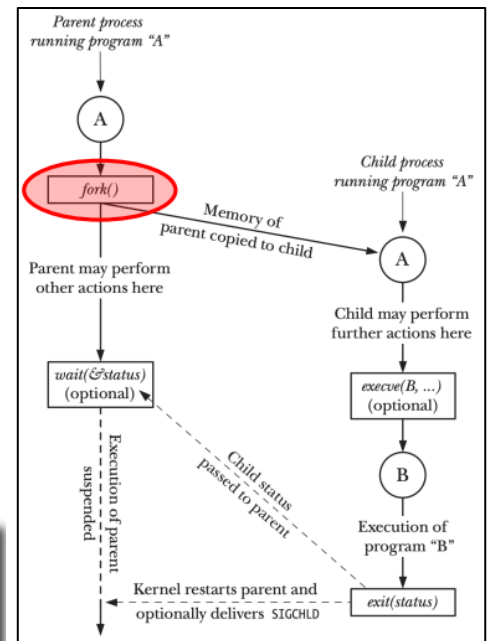
La creazione di processi può essere uno strumento utile per suddividere un compito.

Per esempio, un server di rete può ascoltare le richieste da parte dei client e creare un nuovo processo figlio per gestire ciascuna richiesta, e nel frattempo continuare a restare in ascolto di ulteriori contatti da parte di altri client.

```
#include <unistd.h>
```

```
pid_t fork(void);
```

```
In parent: returns process ID of child on success, or -1 on error;  
in successfully created child: always returns 0
```



Dopo l'esecuzione della `fork()`, esistono 2 processi e in ciascuno l'esecuzione riprende dal punto in cui la `fork` restituisce.

- Il nuovo processo incomincia l'esecuzione a partire dalla prima istruzione successiva alla `fork` che lo ha creato.
- I due processi eseguono lo stesso testo, ma mantenendo copie distinte di stack, data, heap, buffer I/O.
 - stack, dati, heap, buffer I/O del figlio sono inizialmente esatti duplicati del padre...che, dopo la `fork()`, ogni processo può modificare le variabili in tali segmenti senza influenzare l'altro processo.

All'interno del codice di un programma possiamo distinguere i due processi per mezzo del valore restituito dalla `fork`:

- Nell'ambiente del padre, `fork()` restituisce il PID del figlio appena creato.
 - È utile perché il padre può creare e tenere traccia di vari figli.
 - Per attenderne la terminazione può usare la `wait()`
- Nell'ambiente del figlio la `fork()` restituisce 0.
 - Se necessario il figlio può ottenere, come abbiamo visto prima:
 - il proprio PID con la `getpid()`, e
 - il PID del padre con la `getppid()`.

```
pid_t childPid;
```

```
switch (childPid = fork()) {
```

```
case -1: /* fork() failed */  
/* --- Handle error --- */
```

```
case 0: /* Child of successful fork() comes here */  
/* --- Perform actions specific to child --- */
```

```
default: /* Parent comes here after successful fork() */  
/* --- Perform actions specific to parent --- */  
}
```

Nota Bene / Attenzione:

All'esecuzione della `fork()`, il figlio riceve duplicati di tutti i descrittori di file del padre. Quindi: **gli attributi di un file aperto sono condivisi fra genitore e figlio**.

Per esempio, se il figlio aggiorna l'offset del file, tale modifica è visibile attraverso il corrispondente descrittore nel padre.

Problema: Quale processo sarà eseguito?

Dopo una `fork()`, è indeterminato quale dei due processi sarà scelto per ottenere la CPU.

In programmi scritti male, questa indeterminatezza può causare errori

Se invece abbiamo bisogno di garantire un particolare ordine di esecuzione, è necessario utilizzare una qualche tecnica di sincronizzazione (ad esempio: i semafori)

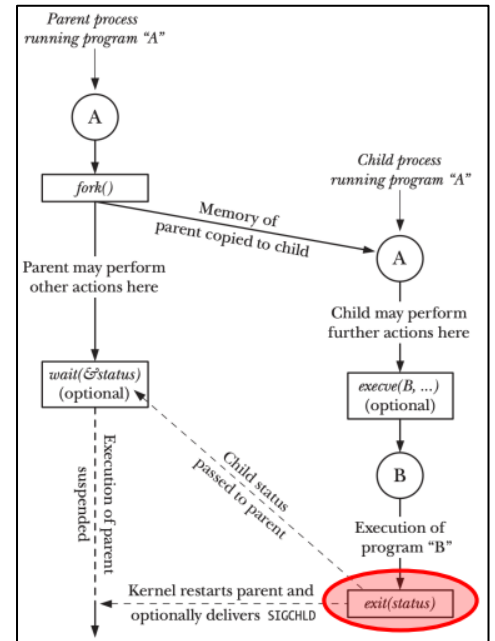
Terminazione dei Processi: `exit()`

La syscall `exit(status)` termina un processo, rendendo le risorse utilizzate dal processo (memoria, descrittori dei file aperti, etc.) nuovamente disponibili per essere allocate dal kernel.

L'argomento `status` è un intero che descrive lo stato di terminazione del processo: utilizzando la system call `wait()` il processo padre può risalire a tale `status`.

Più nello specifico, Quando un processo termina:

- Le funzioni che sono state registrate come *handler* con la funzione `atexit()` vengono eseguite in ordine inverso rispetto alla registrazione
- Tutti gli stream, come ad esempio i files, sono svuotati e chiusi
- Un segnale di `SIGCHILD` è mandato al processo padre
- Ogni figlio (orfano) del processo appena terminato è assegnato ad un nuovo processo padre, il processo *init* (...)
- Le risorse (memoria, descrittori di file aperti) sono rilasciate
- Il valore di `status` in uscita viene memorizzato (...)



Un processo può essere terminato da:

1. La system call `exit(status)`
2. La ricezione di un segnale, ad esempio inviato premendo CTRL+C.
 - a. **ATTENZIONE:** nel caso succeda questo, le funzioni registrate con `atexit` non sono invocate

Il valore di ritorno di `status` può dare le informazioni sul motivo della terminazione del processo:

- Il suo valore può essere compreso tra 0 e 255.
- Due macro sono definite in `stdlib.h` (e dovrebbero essere usate):
 - `EXIT_SUCCESS` (di solito 0)
 - `EXIT_FAILURE` (di solito 1)

Un processo può terminare in due modi principali:

1. **Terminazione Inaspettata**, causata dalla ricezione di un segnale, la cui azione di default è di terminare il processo
2. In alternativa, un processo può **Terminare Normalmente**

```
#include <unistd.h>

void _exit(int status);
```

Usando la syscall `_exit(status)`:

- a. L'argomento `status` definisce lo stato di terminazione del processo, che sarà disponibile al processo padre quando egli chiamerà la funzione `wait()`
- b. Sebbene sia definito come un intero, solamente gli otto bit più significativi sono usati.
- c. Per convenzione:
 - i. un valore di terminazione di 0 indica che il processo è **Terminato con Successo**, mentre
 - ii. un valore diverso da 0 una **Terminazione Inaspettata**

Di solito, il codice dei programmi non contiene la chiamata alla funzione `_exit()`, ma contengono invece la chiamata a `exit()`.

Le azioni eseguite dalla `exit()` sono:

- Gli exit handlers (le funzioni registrate con `atexit()` e `on_exit()`) sono invocate in ordine inverso;
- I buffer dello stream stdio stream vengono svuotati,
- La funzione `_exit()` viene invocata, usando il valore di `status` passato come parametro.

```
#include <unistd.h>

void exit(int status);
```

Monitoraggio dei Processi: wait(&status)

La syscall `wait(&status)` ha due fini:

1. se un figlio non ha ancora concluso la propria esecuzione chiamando una `exit()`, la `wait()` **sospende l'esecuzione del processo chiamante finché uno dei figli non ha terminato la propria esecuzione**; quindi, è uno strumento di **sincronizzazione**.
2. dopo la terminazione del figlio, **lo stato di terminazione del figlio è restituito nell'argomento `status` della `wait()`**.

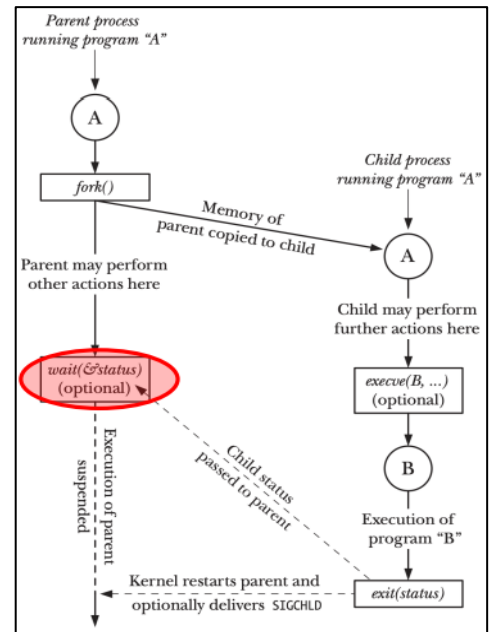
```
#include <sys/wait.h>

pid_t wait(int *status);

Returns process ID of terminated child, or -1 on error
```

La syscall `wait()` restituisce:

- il PID del figlio che ha terminato la propria esecuzione.
- -1, in caso di errore.
 - Un possibile errore è che il processo chiamante potrebbe non avere figli, il che è indicato dal valore `ECHILD` di `errno`.



Possiamo utilizzare questo ciclo per attendere la terminazione di tutti i figli di un processo:

```
while ((childPid = wait(NULL)) != -1)
    continue;

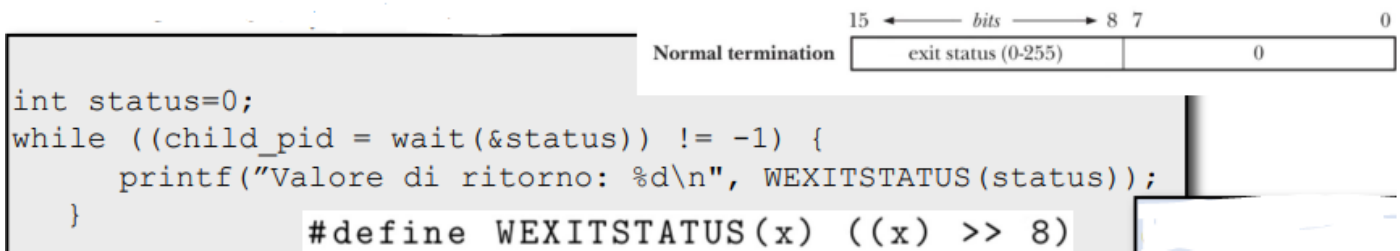
if (errno != ECHILD) // errore inatteso
    errExit("wait"); // gestione errore...
```

Limite: non è possibile attendere la terminazione di uno specifico figlio.

Il parametro `status` a cui punta la `wait()`, viene usato per comunicare al processo padre lo stato di terminazione del processo figlio (che ha invocato la `exit()`).

ATTENZIONE: `status` ha una particolare codifica:

- La macro `WEXITSTATUS(status)` viene utilizzata per estrarre il valore di ritorno del processo figlio; solo gli 8 bit più significativi.



Monitoraggio dei Processi: waitpid()

Limiti della wait():

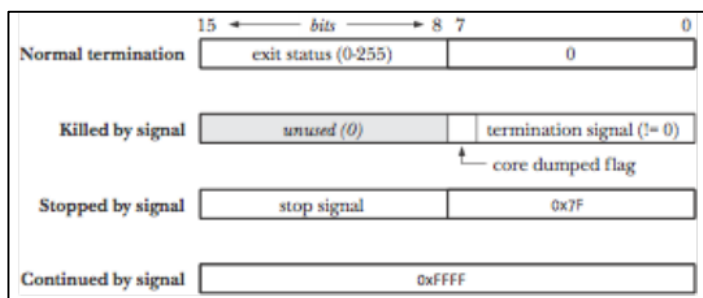
- Se un processo padre ha creato vari figli, non è possibile attendere la terminazione di un particolare figlio;
- Se nessun figlio ha già terminato, la wait() si blocca.
 - In alcuni casi è preferibile eseguire una *nonblocking wait*, in modo da ottenere immediatamente l'informazione che nessun figlio ha ancora terminato la propria esecuzione.
- Con la wait() è possibile avere informazioni sui figli che hanno terminato. Non è invece possibile ricevere notifiche quando un figlio è bloccato da un segnale (come SIGSTOP) o quando un figlio stopped è risvegliato da un segnale (come SIGCONT).

Tutti questi problemi vengono risolti con la waitpid() :

```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);

//Returns process ID of child, 0, or -1 on error
```

- **L'argomento pid permette di selezionare il figlio da aspettare**, secondo queste regole:
 - se `pid > 0`, attendi per il figlio con quel pid;
 - se `pid == 0`, attendi per qualsiasi figlio nello stesso gruppo di processi del chiamante (padre);
 - se `pid < -1`, attendi per qualsiasi figlio il cui process group è uguale al valore assoluto di pid;
 - se `pid == -1`, attendi per un figlio qualsiasi.
 - la chiamata `wait(&status)` è equivalente a `waitpid(-1, &status, 0)`.
- **L'argomento options è una bit mask che può includere** (in OR) zero o più dei seguenti **flag**:
 - WUNTRACED: oltre a restituire info quando un figlio termina, **restituisce informazioni quando il figlio viene bloccato da un segnale**.
 - WCONTINUED: **restituisce informazioni anche nel caso il figlio sia stopped e venga risvegliato da un segnale SIGCONT**.
 - WNOHANG: **se nessun figlio specificato da PID ha cambiato stato, restituisci immediatamente, invece di bloccare il chiamante**.
 - In questo caso, il valore di ritorno di `waitpid()` è 0.
 - Se il processo chiamante non ha figli con il PID richiesto, `waitpid()` fallisce con l'errore ECHILD.
- **Il valore status**, restituito anche da `waitpid()`, **ci consente di distinguere fra i diversi eventi per il figlio**:
ATTENZIONE: `status` ha una particolare codifica: solo gli ultimi 2 Byte sono significativi... e il modo in cui questi 2 byte sono scritti dipende da quale è evento è occorso per il figlio:



L'header file `<sys/wait.h>` definisce un insieme standard di macro che possono essere utilizzate per interpretare `status`.

Applicate allo `status` restituito da `wait()` o `waitpid()`, solo una delle seguenti macro restituirà true:

- `WIFEXITED(status)`, restituisce **true se il processo figlio è terminato normalmente**.
 - In questo caso la macro `WEXITSTATUS(status)` restituisce l'exit status del processo figlio.
- `WIFSIGNALED(status)`, restituisce **true se il figlio è stato ucciso da un segnale**.
 - In questo caso, la macro `WTERMSIG(status)` restituisce il numero del segnale che ha causato la terminazione del processo.
- `WIFSTOPPED(status)`, restituisce **true se il figlio è stato bloccato da un segnale**.
 - In questo caso, la macro `WSTOPSIG(status)` restituisce il numero del segnale che ha bloccato il processo.
- `WIFCONTINUED(status)`, **restituisce true se il figlio è stato risvegliato da un segnale SIGCONT**.

Problema: Processi Orfani & Zombie

Caso 1: Il Padre termina prima del Figlio; Figlio Orfano

Chi diventa il padre di un processo figlio orfano? Il Figlio Orfano è adottato da *init*, il progenitore di tutti i processi, il cui PID è 1.

- In altre parole, dopo che il genitore di un processo figlio termina, una chiamata a `getppid()` restituirà il valore 1 (l'id dipende dal SO). Questo può essere utile per capire se il vero padre di un processo figlio è ancora vivo.

Caso 2: Figlio Zombie

Cosa capita a un figlio che termina prima che il padre abbia avuto modo di eseguire una `wait()`?

- Sebbene il figlio abbia terminato, il padre dovrebbe poter avere la possibilità di eseguire una `wait()` in un momento successivo per determinare come è terminato il figlio.
- Il kernel garantisce questa possibilità trasformando il figlio in uno Zombie.
 - Gran parte delle risorse gestite da un figlio sono rilasciate al sistema per essere assegnate ad altri processi.
 - L'unica parte del processo che resta è un'entry nella "Tabella dei Processi" che registra il PID del figlio, il `termination status`, e le statistiche sull'utilizzo delle risorse.

ATTENZIONE ai Processi Zombie:

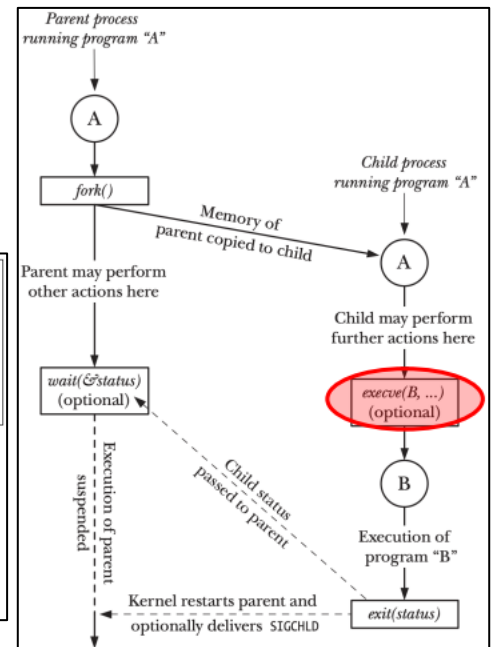
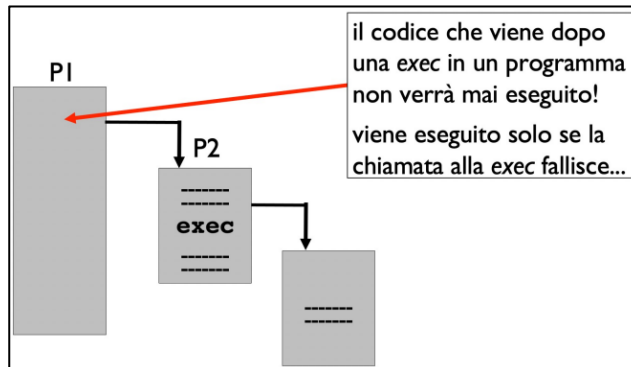
- **Un Processo Zombie non può essere ucciso da un segnale, neppure SIGKILL.** Questo assicura che il genitore possa sempre eventualmente eseguire una `wait()`.
- Quando il padre esegue una `wait()`, il kernel rimuove lo Zombie, dal momento che l'ultima informazione sul figlio è stata fornita all'interessato.
 - Poiché gli zombie non possono essere uccisi da un segnale, l'unico modo per rimuoverli dal sistema è uccidere il loro padre (o attendere la sua terminazione). A quel momento gli zombi possono essere adottati da *init* e rimossi.
- Se il genitore termina senza fare la `wait()`, il processo *init* adotta il figlio ed esegue automaticamente una `wait()`, rimuovendo dal sistema il processo zombie.
- Se un genitore crea un figlio, ma fallisce la relativa `wait()`, un elemento relativo allo zombie sarà mantenuto indefinitamente nella tabella dei processi del kernel.
- Se il numero degli zombie cresce eccessivamente, gli zombie possono riempire la tabella dei processi, e questo impedirebbe la creazione di altri processi.

Esecuzione dei Processi: `execve()`

La syscall `execve(pathname, argv, envp)` carica un nuovo programma nella memoria del processo.

- con questa operazione, il vecchio programma è abbandonato, e lo stack, i dati, e lo heap del processo sono sostituiti da quelli del nuovo programma. **N.B: il processo rimane lo stesso!**

Dopo avere eseguito l'inizializzazione del codice, il nuovo programma inizia l'esecuzione dalla propria funzione `main()`.



```
#include <unistd.h>
int execve(const char *pathname, char *const argv[],
            char *const envp[]);

//Never returns on success; returns -1 on error
```

L'argomento `pathname` contiene il pathname del programma che sarà caricato nella memoria del processo.

- L'argomento `argv` specifica gli argomenti della linea di comando da passare al nuovo programma.
 - Si tratta di una lista di puntatori a stringa, terminati da puntatore a NULL.
 - Il valore fornito per `argv[0]` corrisponde al nome del comando.
 - Tipicamente, questo valore è lo stesso del basename (i.e., l'ultimo elemento) del pathname.
- L'ultimo argomento, `envp`, specifica la lista environment list per il nuovo programma.
 - è una lista di puntatori a stringhe (terminata da ptr a NULL) nella forma `name=value`.

Valori Restituiti dalla `execve`:

- Poiché sostituisce il programma che la ha chiamata, **una chiamata di `execve()` che va a buon fine non restituisce**. Non abbiamo quindi bisogno di controllare il valore di ritorno di `execve()`; sarà sempre `-1`.
- Il fatto che abbia restituito un qualche valore ci informa che è occorso un errore, e come sempre è possibile utilizzare `errno` per determinarne la causa**. Fra gli errori che possono essere restituiti in `errno`:
 - `EACCES`; il `pathname` non si riferisce a un file eseguibile, o non è ricercabile (...).
 - `ENOENT`; il file riferito dal `pathname` non esiste.
 - `ENOEXEC`; il file riferito dal `pathname` non è effettivamente eseguibile.
 - `ETXTBSY`; il file riferito dal `pathname` è aperto in scrittura da un altro processo.

Esistono altre alternative a `execve` che però sono EQUIVALENTI:

```
#include <unistd.h>
int execle(const char *pathname, const char *arg, ...
            /* , (char *) NULL, char *const envp[] */);
int execlp(const char *filename, const char *arg, ...
            /* , (char *) NULL */);
int execvp(const char *filename, char *const argv[]);
int execv(const char *pathname, char *const argv[]);
int execl(const char *pathname, const char *arg, ...
            /* , (char *) NULL */);

//None of the above returns on success; all return -1 on error
```

Nota Bene/Attenzione:

Per default, tutti i descrittori di file aperti da un programma che chiama `exec()` restano aperti e sono pertanto disponibili per il nuovo programma.

Questo è spesso utile, poiché questi file sono automaticamente disponibili per il nuovo programma, senza che questo debba sapere i loro nomi e/o aprirli.

Esecuzione dei Processi: `system()`

La funzione `system()` permette di chiamare un programma per eseguire un comando di shell arbitrario.

Detto in altre parole:

la funzione `system()` crea un processo figlio che invoca una shell per eseguire il comando `command`.

```
#include <stdlib.h>
int system(const char *command);
```

Esempio di chiamata di `system()`: `system("ls -lt | wc -l");`

Valore di ritorno di `system()`:

- se `command` è un NULL pointer, `system()` restituisce:
 - un valore diverso da 0 se una shell è disponibile, e
 - 0 se nessuna shell è disponibile.
- se non è stato possibile creare un processo figlio o il suo stato di terminazione non è stato ricevuto,
 - `system()` restituisce -1.
- se non è stato possibile eseguire la shell nel processo figlio,
 - `system()` restituisce un valore come se la shell del processo figlio avesse terminato con la chiamata `_exit(127)`.
- se tutte le `system` calls hanno avuto successo,
 - `system()` restituisce lo stato di terminazione della shell figlia utilizzata per eseguire il comando: lo status di terminazione di una shell è lo status di terminazione dell'ultimo comando eseguito.

Il vantaggio principale offerto da `system()` è la semplicità d'uso:

- non dobbiamo gestire i dettagli relativi alle chiamate di `fork()`, `exec()`, `wait()`, e `exit()`.
- la gestione degli errori e dei segnali è affidata a `system()` per nostro conto.
- poiché `system()` utilizza la shell per eseguire un comando, il processamento legato alle sostituzioni, redirezioni è effettuato sul comando prima che esso sia eseguito.

Il principale costo della `system()` è l'inefficienza:

- Eseguire un comando usando `system()` richiede la creazione di almeno 2 processi (uno per la shell e uno o più per i comandi eseguiti), ciascuno dei quali esegue una `exec()`.
 - Se l'efficienza o la velocità sono richiesti, è preferibile utilizzare chiamate `fork()` e `exec()` per eseguire il programma desiderato.
-

Extra: gdb - debugger

`gdb` è un software che permette di effettuare un debug di un programma.

1. Prerequisiti:

- compilare il programma con i flags:
 - `-g`, per aggiungere informazioni aggiuntive utili al debug
 - `-O0` per disabilitare tutte le ottimizzazioni del codice

```
gcc -g -O0 programma.c
```

2. Per eseguire un programma con `gdb`:

```
gdb programma
```

N.B: Non inserire eventuali parametri del programma; verranno inseriti successivamente.

Dopo la sua esecuzione, appare il prompt (`gdb`)

A questo punto, i comandi di `gdb` possono essere inseriti:

- `help` elenca i comandi a disposizione
- `list` mostra una porzione del codice sorgente del programma
 - `list <numero-linea>` mostra il codice sorgente intorno alla linea indicata
- `break <numero-linea>` inserisce un breakpoint alla linea indicata
- `info b` mostra i breakpoint definiti (ogni breakpoint è definito da un `<id>`)
- `del <id>` elimina un breakpoint
- `run` , esegue il programma
 - `run <parametri>` esegue il programma con i parametri specificati
- `next` esegue una linea di codice. Se si tratta di una funzione, la esegue
- `step` esegue una linea di codice. Se si tratta di una funzione, entra nel codice della funzione
- `cont` prosegue l'esecuzione del programma fino al prossimo breakpoint
- `print <espressione>` valuta l'espressione e ne stampa il valore
- `display <espressione>` valuta l'espressione e ne stampa il valore ad ogni step
- `quit` termina l'esecuzione di `gdb`

E' possibile connettere `gdb` ad un processo in esecuzione con `gdb -p <PID>`

- Se il processo sta eseguendo una system call, potrebbe essere necessario eseguire come *superuser* con: `sudo gdb -p <PID>`

Domanda: se sto facendo debug di un processo che successivamente esegue una fork, quale dei due processi sarà oggetto di debug? In `gdb` utilizzare uno dei due comandi:

- `set follow-fork-mode parent`
- `set follow-fork-mode child`